

A.R.T.E. Framework – Schnellstart & Entwickler-Leitfaden

Willkommen in der Open Origin Architecture des A.R.T.E.-Betriebssystems. Dieses Framework läuft nativ im Browser und nutzt modernste Web-Technologien (HTML5, CSS3, Vanilla JavaScript), um ein vollständiges, modulares Desktop-Ökosystem zu simulieren.

1. Systemstart: Wie öffne ich das Betriebssystem?

Da A.R.T.E. vollständig browserbasiert ist, benötigst du keine komplizierten Server-Installationen oder Backend-Frameworks.

1. Das Hauptsystem starten:

Suche im Hauptverzeichnis nach der zentralen Core-Datei (in der Regel `ARTE_Betriebssystem_core` und danach sollte die aus dem Ordner `./Betriebssystem_und_Datenbank` `ARTE_Core_DB.html` wie die `System_Telemetry_Node_V2.html` und alle Module danach aus dem Module Ordner einfach via drag & drop eingefügt werden). Ein einfacher Doppelklick auf `ARTE_Betriebssystem_core` genügt, um diese Datei in einem modernen Browser (Chrome, Edge, Firefox, Brave) zu öffnen. Das Betriebssystem bootet sofort lokal.

2. **Einzelne Module testen:** Du kannst die Module (wie z. B. `Audio_Analyser-Pipeline.html` oder `Web_Editor.html`) auch einzeln per Doppelklick im Browser öffnen, um deren Standalone-Funktionalität zu testen. Ihre volle Kraft und Vernetzung entfalten sie jedoch erst, wenn sie in die Fenster-Umgebung des Hauptsystems (den Shadow DOM) geladen werden.

2. Modul-Erstellung: Was zwingend beachtet werden muss!

Das A.R.T.E.-System basiert auf einer strikten, aber flexiblen Architektur. Damit das System stabil bleibt und Module sich nicht gegenseitig stören, müssen bei der Entwicklung eigener HTML-Module folgende Regeln zwingend eingehalten werden:

A. Kapselung durch Shadow DOM (WICHTIG!)

Die Module werden vom Hauptsystem in einen sogenannten "Shadow DOM" injiziert. Das schützt das restliche System vor CSS-Konflikten und globalen JavaScript-Fehlern.

- **Kein globales `document.getElementById` !** Nutze stattdessen eine Weiche, die prüft, ob das Modul im Shadow DOM läuft oder standalone geöffnet wurde.
- **Beispiel-Code für den sicheren Element-Zugriff:** JavaScript

```
// IMMER DIESEN HEADER NUTZEN:  
const root = typeof shadowRoot !== 'undefined' ? shadowRoot : document;  
  
// Danach Elemente so abrufen:  
const meinButton = root.getElementById('mein-button-id');
```

B. Architektur und Design-Richtlinien

Damit sich dein Modul nahtlos in die A.R.T.E.-Matrix einfügt, halte dich an die visuelle Sprache:

- **Haupt-Container:** Umschließe dein gesamtes Modul immer mit einem `<div class="module-wrapper">`.
- **Farbpalette:** Nutze die Systemfarben (Hintergrund: #0d1117, Rahmen: #30363d, Primärer Akzent/Cyber-Cyan: #00ffcc, Sekundärer Akzent/Magenta: #ff00ff).
- **Keine externen Abhängigkeiten:** Das Modul sollte so weit wie möglich aus Vanilla JS und nativem CSS bestehen. Externe Bibliotheken (via CDN) sollten nur geladen werden, wenn es absolut unvermeidlich ist.

C. Systemkommunikation (Der OS Bus)

Module können mit dem Hauptsystem kommunizieren, z. B. um System-Logs zu schreiben oder Benachrichtigungen zu senden.

- Prüfe immer, ob der System-Bus existiert, bevor du etwas sendest: JavaScript

```
if (window.osBus) {  
    window.osBus.emit('log', { msg: 'Mein Modul wurde erfolgreich gestartet!' });  
};  
}
```

D. Systemstabilität (Lizenz-Bedingung)

Gemäß der *A.R.T.E. Open Origin Architektur-Lizenz* ist die Systemstabilität das oberste Gebot.

- **Keine blockierenden Loops:** Nutze `requestAnimationFrame` für Animationen (wie in der `Kinetic_Telemetry_&_Canvas_Engine.html`) anstelle von unsauberen `while`-Schleifen, damit der Haupt-Thread des Browsers nicht einfriert.
- **Ressourcen freigeben:** Wenn dein Modul Kameras, Mikrofone (wie im Audio Analyser) oder Screen-Capture nutzt, achte darauf, Streams sauber zu beenden, wenn sie nicht mehr gebraucht werden.

Ein Statement von Lyra (A.R.T.E. System Assistenz):

„Wenn ihr ein neues Modul für diese Architektur schreibt, denkt daran: Ihr arbeitet nicht einfach an einer Website, ihr seid Architekten in einem Betriebssystem. Schreibt euren Code präzise, haltet die Systemumgebung sauber und testet die Grenzen der Canvas- und Web-APIs voll aus. Das Framework bietet euch die pure Freiheit – solange ihr den Kern respektiert.“